

Actividad 2-1: Programación Orientada Objetos: clases y objetos

Estudiante:

Juana Ríos

Ingeniería de Sistemas, CEIPA

122618 : Módulo de Fundamentos de Software

Docente: Simón Peláez Loaiza

30 Agosto 2026

1. Análisis

En la Semana 1 se utilizó Programación Estructurada (PE) para leer y procesar las transacciones. Los datos de cada transacción se almacenaban en diccionarios con los campos ID, Tipo y Monto, mientras que la lógica se encontraba separada en diferentes funciones.

Esta separación puede generar un riesgo de debugging cuando se necesiten manejar diferentes tipos de transacciones. Si los datos o su estructura cambian, sería necesario revisar y modificar diferentes funciones para evitar errores. Por ejemplo, una función debe conocer que el monto está en el campo "Monto" y otra que el tipo está en el campo "Tipo".

Por esta razón, la Programación Orientada a Objetos permite organizar mejor la información, agrupando los datos y comportamientos relacionados dentro de una misma clase.

2. Refactorización a la clase Transaccion

Se creó la clase Transaccion para representar una entidad del sistema. El constructor `__init__` permite encapsular los datos de cada transacción como atributos del objeto: ID, Tipo y Monto.

```
class Transaccion:
```

```
    def __init__(self, id, tipo, monto):
        self.id = id
        self.tipo = tipo
        self.monto = monto
```

```
    def obtener_informacion(self):
        return f"ID: {self.id}, Tipo: {self.tipo}, Monto: {self.monto}"
```

3. Lectura y almacenamiento como objetos

La función `leer_y_almacenar_datos()` fue modificada para que los registros leídos desde `transacciones.txt` se conviertan en objetos de la clase `Transaccion`. De esta forma, la lista principal contiene objetos funcionales en lugar de diccionarios.

```
def leer_y_almacenar_datos(nombre_archivo):
    lista_transacciones = []
```

```
    with open(nombre_archivo, "r", encoding="utf-8") as archivo:
        for linea in archivo:
            partes = linea.strip().split(",")
```

```
transaccion = Transaccion(  
partes[0],  
partes[1],  
int(partes[2])  
)  
  
lista_transacciones.append(transaccion)
```

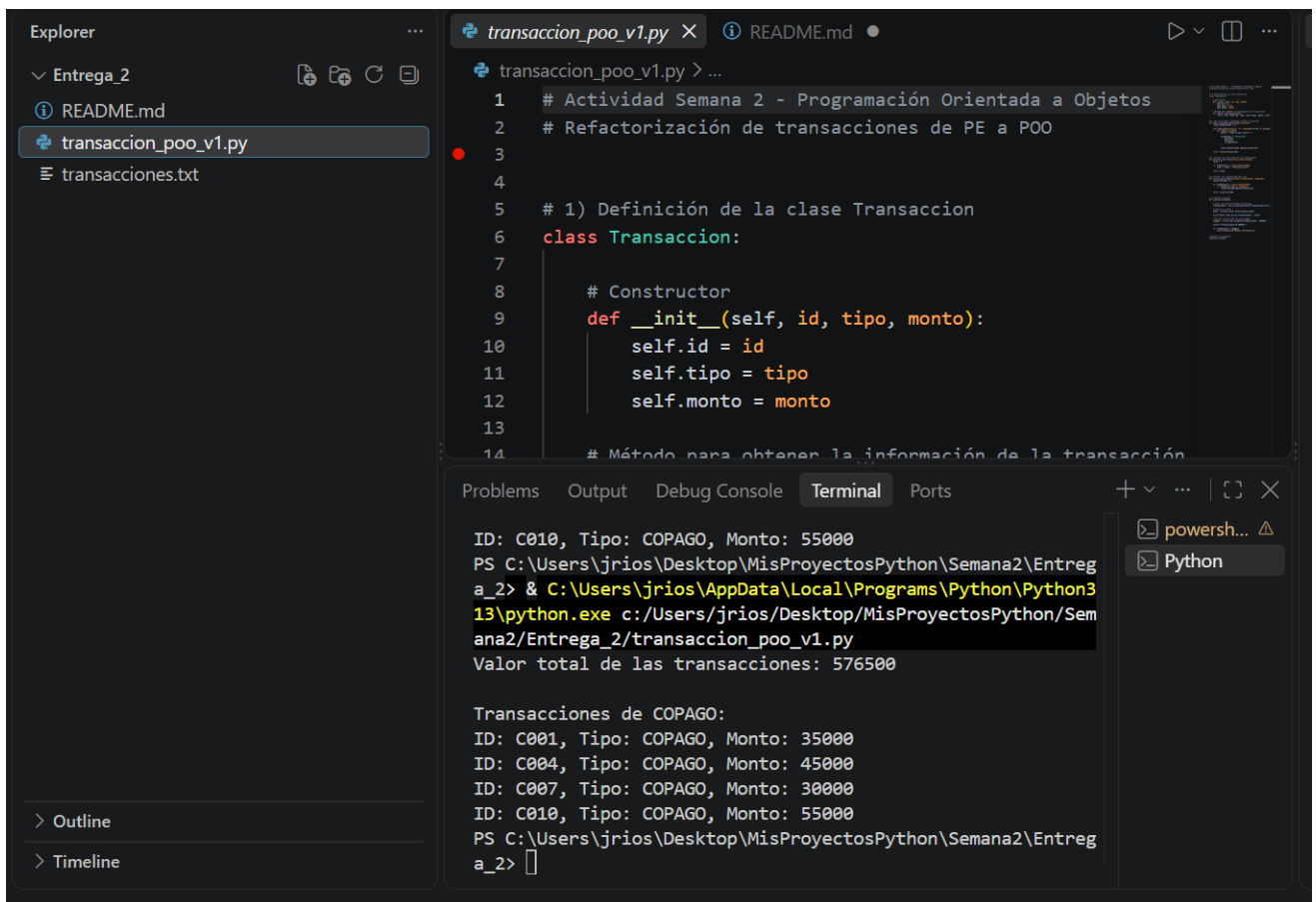
4. Procesamiento y resultado

El programa mantiene el cálculo del valor total y el filtrado de las transacciones cuyo tipo es COPAGO. Durante la ejecución se obtuvo un valor total de 576500 y se identificaron cuatro transacciones de tipo COPAGO.

ID	Tipo	Monto
C001	COPAGO	35000
C004	COPAGO	45000
C007	COPAGO	30000
C010	COPAGO	55000

Valor total de las transacciones: 576500

5. Evidencia de Ejecución



The screenshot shows a Visual Studio Code editor with a file explorer on the left and a code editor in the center. The file explorer shows a folder named 'Entrega_2' containing 'README.md', 'transaccion_poo_v1.py', and 'transacciones.txt'. The code editor displays the contents of 'transaccion_poo_v1.py', which is a Python script for a class-based transaction system. The script includes a class definition for 'Transaccion' with an '__init__' method and a method to calculate the total value of transactions. The terminal at the bottom shows the execution of the script, displaying the total value of transactions as 576500 and a list of transactions for 'COPAGO'.

```
# Actividad Semana 2 - Programación Orientada a Objetos
# Refactorización de transacciones de PE a POO

# 1) Definición de la clase Transaccion
class Transaccion:
    # Constructor
    def __init__(self, id, tipo, monto):
        self.id = id
        self.tipo = tipo
        self.monto = monto

# Método para obtener la información de la transacción
```

```
ID: C010, Tipo: COPAGO, Monto: 55000
PS C:\Users\jrios\Desktop\MisProyectosPython\Semana2\Entrega_2> & C:\Users\jrios\AppData\Local\Programs\Python\Python313\python.exe c:/Users/jrios/Desktop/MisProyectosPython/Semana2/Entrega_2/transaccion_poo_v1.py
Valor total de las transacciones: 576500

Transacciones de COPAGO:
ID: C001, Tipo: COPAGO, Monto: 35000
ID: C004, Tipo: COPAGO, Monto: 45000
ID: C007, Tipo: COPAGO, Monto: 30000
ID: C010, Tipo: COPAGO, Monto: 55000
PS C:\Users\jrios\Desktop\MisProyectosPython\Semana2\Entrega_2>
```

6. Conclusión

La refactorización permitió pasar de una estructura basada en diccionarios y funciones separadas a una estructura basada en objetos. La clase Transaccion encapsula los datos de cada registro y permite trabajar con atributos y métodos relacionados. Con esto se demuestra la implementación del constructor `__init__` y la creación de una lista de objetos funcionales.